

CSC 250: Foundations of Computer Science I

Spring 2026

Chapter 4: Binary Relations and Inductive Proof

Sakhawat Hossan

Department of Computer Science
University of North Carolina at Greensboro

Section 4.1 – Properties of Binary Relations

Binary relations describe relationships between elements of a set.

They are widely used in:

- databases
- graph theory
- ordering systems
- computer science algorithms

Binary Relation on Q

$Q = \{\text{people at the work event}\}$



Sara



Amy

$R = \{(q_1, q_2) \mid q_1 \text{ and } q_2 \text{ are in } Q$
and have the same hair color}

(Sara, Amy) is in R , because Sara
and Amy are at the work event and
have the same hair color.

Binary Relation

A binary relation R over a set A is defined as a subset of the Cartesian product $R \subseteq A \times A$

If a pair (x, y) belongs to relation R , we write

$$xRy$$

which means x is related to y under relation R .

Example

Let, $A = \{0, 1\}$ Possible relations on A :



Relation	Set of ordered pairs
Empty relation	$\emptyset$
Universal relation	$A \times A = \{(0,0), (0,1), (1,0), (1,1)\}$
Equality relation	$eq = \{(0,0), (1,1)\}$

Reflexive Relation

A relation R on set A is reflexive if every element is related to itself.

$$xRx \text{ for all } x \in A$$

This means every element has a self-loop.

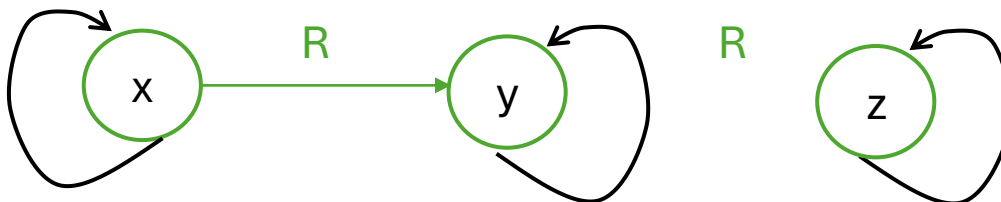
Set Example

Let, $A = \{1, 2, 3\}$

Relation, $R = \{(1, 1), (2, 2), (3, 3), (1, 2)\}$

Since $(1, 1)$, $(2, 2)$, $(3, 3)$ are included, the relation is reflexive.

Graph Example



Irreflexive Relation

A relation R is irreflexive if no element is related to itself.

$$(x, x) \notin R \text{ for all } x \in A$$

No self-loops exist.

Set Example

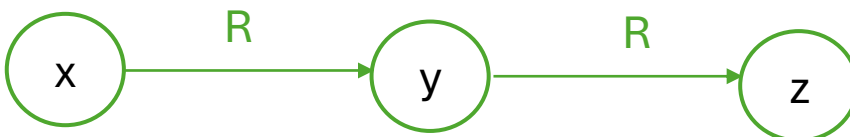
Let, $A = \{1, 2, 3\}$

$$R = \{(1, 2), (2, 3), (1, 3)\}$$

There are no pairs like $(1, 1)$, $(2, 2)$, $(3, 3)$

So, the relation is irreflexive.

Graph Example



Symmetric Relation

A relation R is symmetric if xRy then yRx . Whenever one pair appears, the reverse pair must also appear.

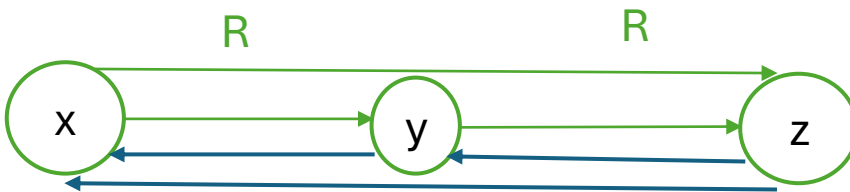
Set Example

Let, $A = \{1, 2, 3\}$

$$R = \{(1, 2), (2, 1), (2, 3), (3, 2)\}$$

Since every pair has its reverse pair, the relation is symmetric.

Graph Example



Antisymmetric Relation

A relation R is antisymmetric if xRy and yRx then $x = y$. This means:

If two elements relate to each other both ways, they must actually be the same element.

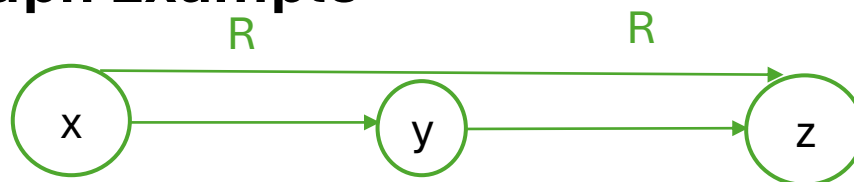
Set Example

Let, $A = \{1, 2, 3\}$

$$R = \{(1, 1), (2, 2), (3, 3), (1, 2), (1, 3), (2, 3)\}$$

There are no pairs like $(2, 1)$ or $(3, 2)$. This relation is antisymmetric.

Graph Example



Transitive Relation

A relation R is transitive if xRy and yRz then xRz

If a connection exists through an intermediate element, the direct connection must also exist.

Set Example

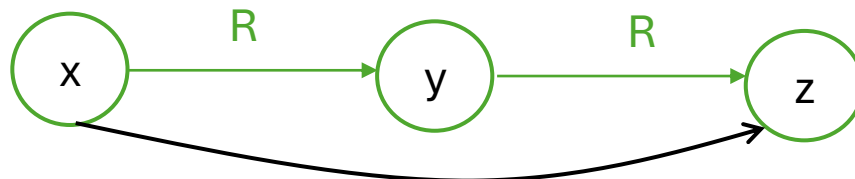
Let, $A = \{1, 2, 3\}$

$$R = \{(1, 2), (2, 3), (1, 3)\}$$

Because $(1, 2)$ and $(2, 3)$ imply $(1, 3)$

the relation is transitive.

Graph Example



Example

Describe the properties that hold for the four sample relations:

1. $\emptyset$.
2. $A \times A$.

Recall the properties:

Reflexive: $(x, x) \in R$ for all $x \in A$

Irreflexive: $(x, x) \notin R$ for all $x \in A$

Symmetric: if $(x, y) \in R$ then $(y, x) \in R$

Antisymmetric: if $(x, y) \in R$ and $(y, x) \in R$ then $x = y$

Transitive: if $(x, y) \in R$ and $(y, z) \in R$ then $(x, z) \in R$

Answers :

1. symmetric, transitive, irreflexive, antisymmetric.
2. reflexive, symmetric, transitive.

1. Empty Relation $R = \emptyset$

Reflexive? No.

Because, reflexive requires

$(0, 0), (1, 1)$

but they are missing.

Irreflexive? Yes.

Irreflexive requires no self pairs, which is true.

Symmetric? Yes.

Symmetric says:

if $(x, y) \in R$ then $(y, x) \in R$

Since there are no pairs, the rule is never violated.

1. Empty Relation $R = \emptyset$

Antisymmetric? Yes.

There are no pairs like

(x, y) and (y, x)

So, the condition holds.

Transitive? Yes.

Transitive requires

$(x, y), (y, z) \Rightarrow (x, z)$

Since there are no pairs, the condition is satisfied.

2. $A \times A$.

Reflexive? Yes.

Because reflexive requires

$$(0, 0), (1, 1)$$

Both are present.

Irreflexive? No.

Irreflexive requires no self pairs. $(x, x) \notin R$

But:

$$(0, 0), (1, 1)$$

exist.

Symmetric? Yes.

Symmetric says:

if $(x, y) \in R$ then $(y, x) \in R$

$(0, 1) \Rightarrow (1, 0)$ exist.

2. $A \times A$.

Antisymmetric? No

If (x, y) and (y, x)

Then $x = y$

But: $(0, 1)$ and $(1, 0)$

exist and $0 \neq 1$

Transitive? Yes.

Transitive requires

$$(x, y), (y, z) \Rightarrow (x, z)$$

Example:

$$(0, 1), (1, 0) \Rightarrow (0, 0)$$

All such pairs exist in $A \times A$.

Composition of Relations

If R and S are relations, then:

$$R \circ S = \{(x, z) \mid xRy \text{ and } ySz \text{ for some } y\}$$

Think of it like a path: $x \text{ --}R\text{--} \rightarrow y \text{ --}S\text{--} \rightarrow z$

Then composition gives: $x \text{ -----} \rightarrow z$

Example: $\text{eq} \circ \text{less} = \text{less}$

Given: $\text{eq} = \text{equality } (x = y)$, $\text{less} = \{(0, 1)\}$

We need:

$$xRy \text{ and } ySz$$

So: $xRy \rightarrow x = y$

$ySz \rightarrow y < z$

Substitute $x = y$:

this becomes: $x < z$

Path Problem (*Is there a path from i to j ?*)

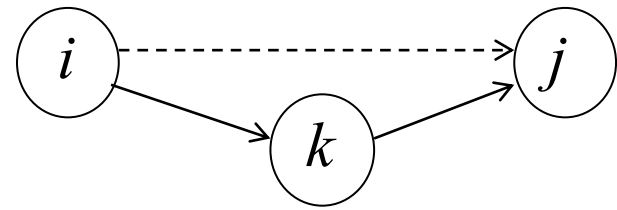
Warshall's Algorithm

Used to compute the transitive closure of a relation. Add all missing pairs so that: *R becomes transitive*

Key Idea:

If: xRy and yRz

then add: xRz



Relation: $A = \{1, 2, 3, 4\}$

Let $R = \{(1, 2), (2, 3), (3, 4)\}$.

We can represent R as an adjacency matrix M where M_{ij} is 1 if $i R j$ and 0 otherwise. If we want an answer to our question, it would be nice to have the adjacency matrix for $t(R)$. Then we could simply check whether $M_{ij} = 1$.

Warshall's Algorithm Example

Matrix:

$$M = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Warshall's algorithm computes the matrix for $t(R)$. It constructs edge (i, j) if it finds edges (i, k) and (k, j) , as pictured.

for $k := 1$ to n **for** $i := 1$ to n **for** $j := 1$ to n **do**
if $M_{ik} = M_{kj} = 1$ **then** $M_{ij} := 1$.

Example. The following trace shows M after each k -loop, where the rightmost M contains the adjacency matrix of $t(R)$.

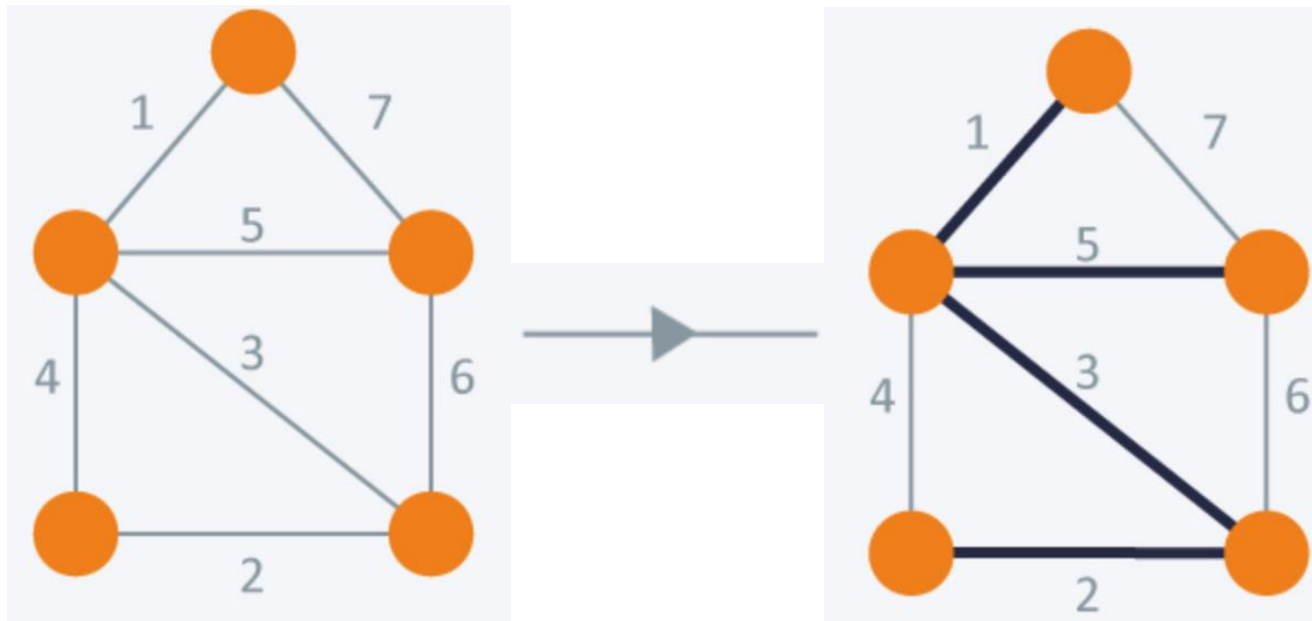
$k = 1$ (no change) $\begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$	→	$k = 2$ $M_{13} := (M_{12} = M_{23} = 1)$ $\begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$	→	$k = 3$ $M_{14} := (M_{13} = M_{34} = 1)$ $M_{24} := (M_{23} = M_{34} = 1)$ $\begin{bmatrix} 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$	→	$k = 4$ (no change) $\begin{bmatrix} 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$
--	---	--	---	---	---	--

Minimum Spanning Tree (MST)

A Minimum Spanning Tree (MST) is:

A subset of edges of a connected, weighted, undirected graph that:

- connects all vertices
- has no cycles (it is a tree)
- has the minimum total edge weight

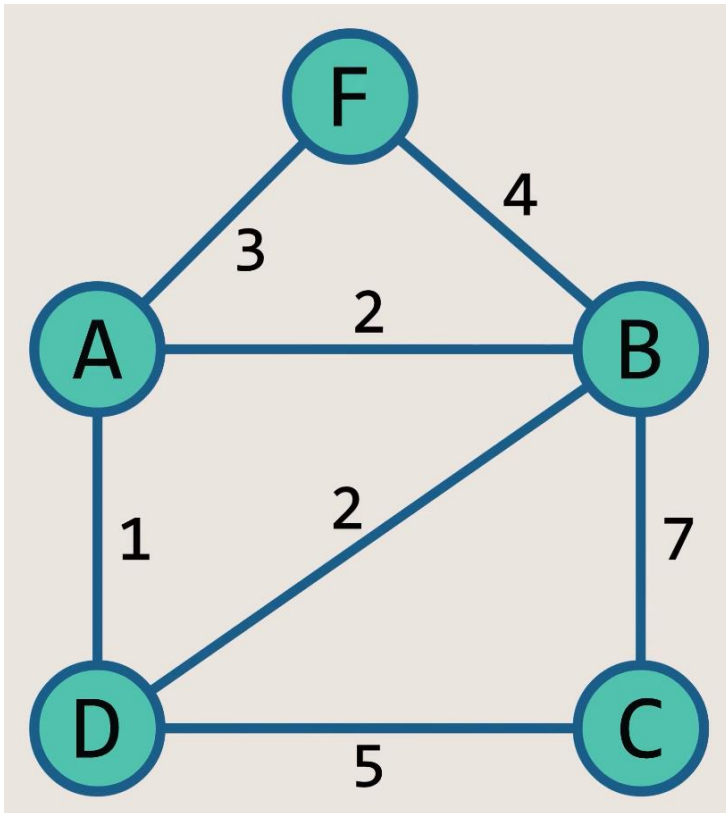


Kruskal's Algorithm

Below are the steps for finding MST using Kruskal's algorithm:

- Sort all the edges in an ascending order of their weight.
- Pick the smallest edge. That connect an unvisited edge.
Check if it forms a cycle with the spanning tree formed so far.
If the cycle is not formed, include this edge. Else, discard it. It uses the [Disjoint Sets](#) to detect cycles.
- Repeat step 2 until there are $(V-1)$ edges in the spanning tree.

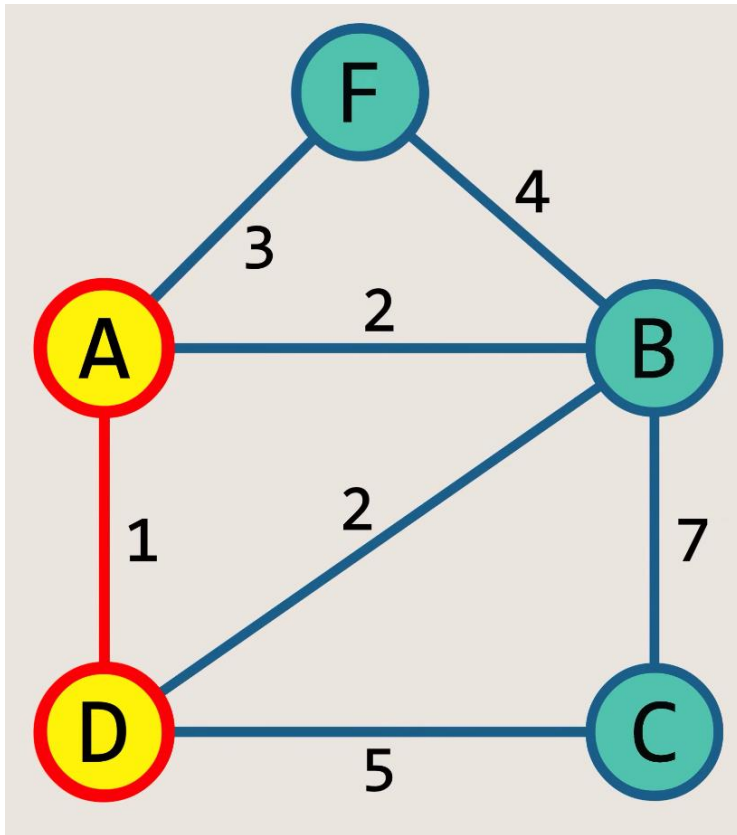
Kruskal's Algorithm



Step 1: Sort the edge weights:

1 2 2 3 4 5 7

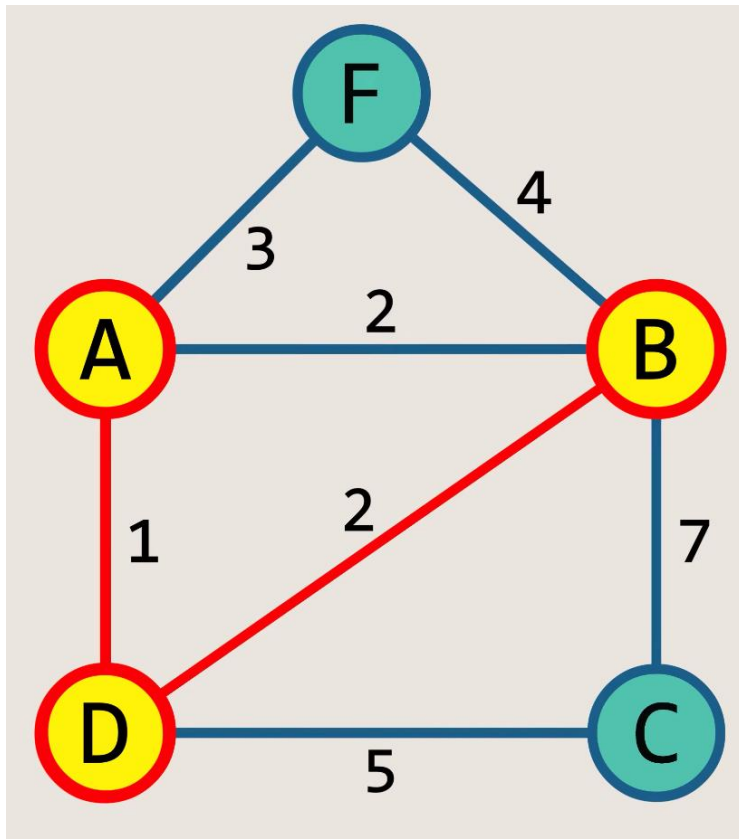
Kruskal's Algorithm



Step 2: Select the smallest edge weight (1) and add to the spanning tree

1 2 2 3 4 5 7

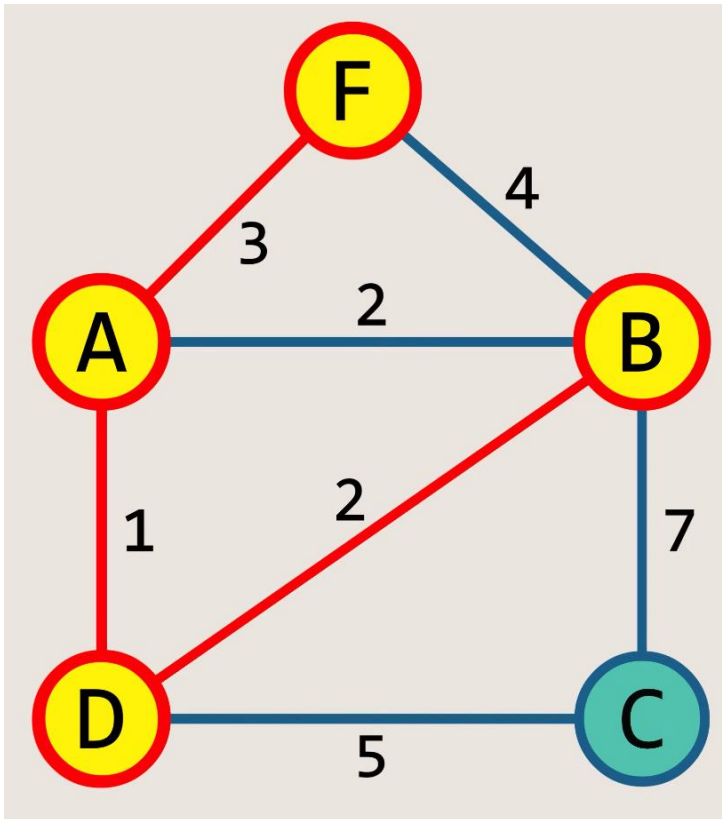
Kruskal's Algorithm



Step 3: Select the smallest edge weight (2) and add to the spanning tree. We have two edges with the weight 2 so select either of them. We can not add both because it will form a cycle.

1 2 2 3 4 5 7

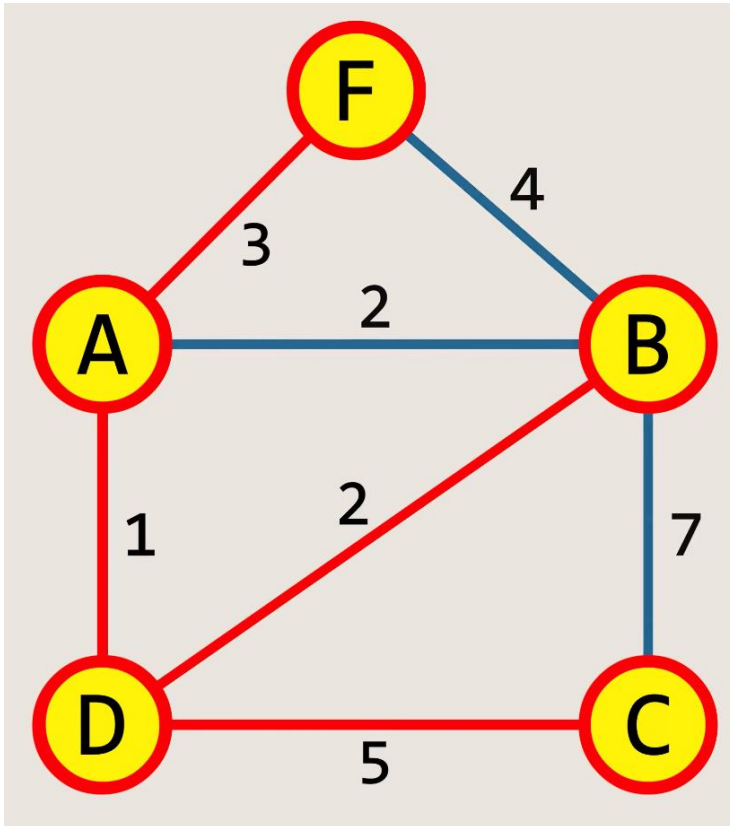
Kruskal's Algorithm



Step 4: Select the smallest edge weight and add to the spanning tree

1 2 2 3 4 5 7

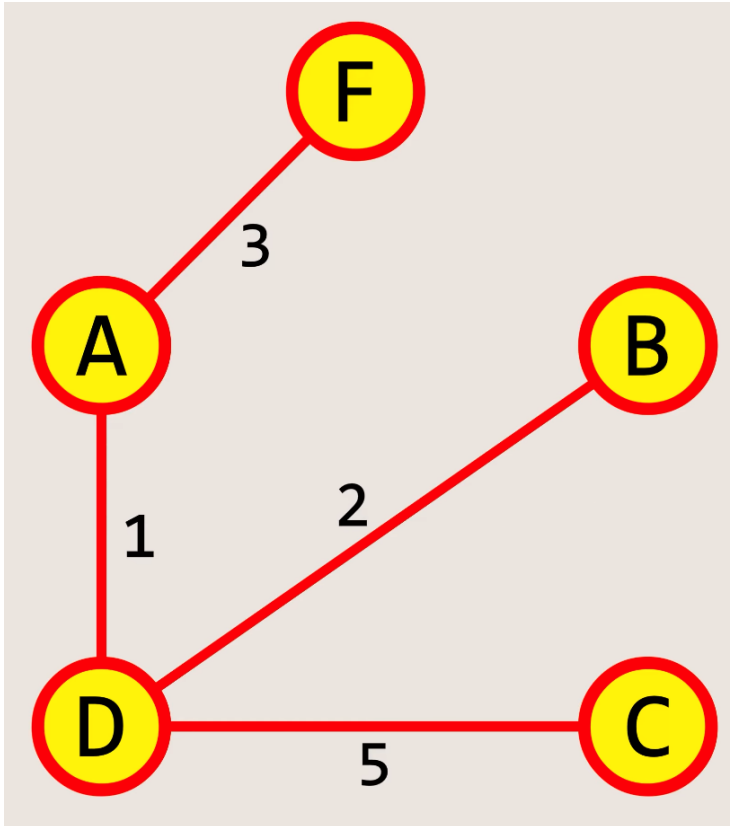
Kruskal's Algorithm



Step 5: Select the smallest edge weight (5) and add to the spanning tree. We skipped 4 because, it will create a cycle.

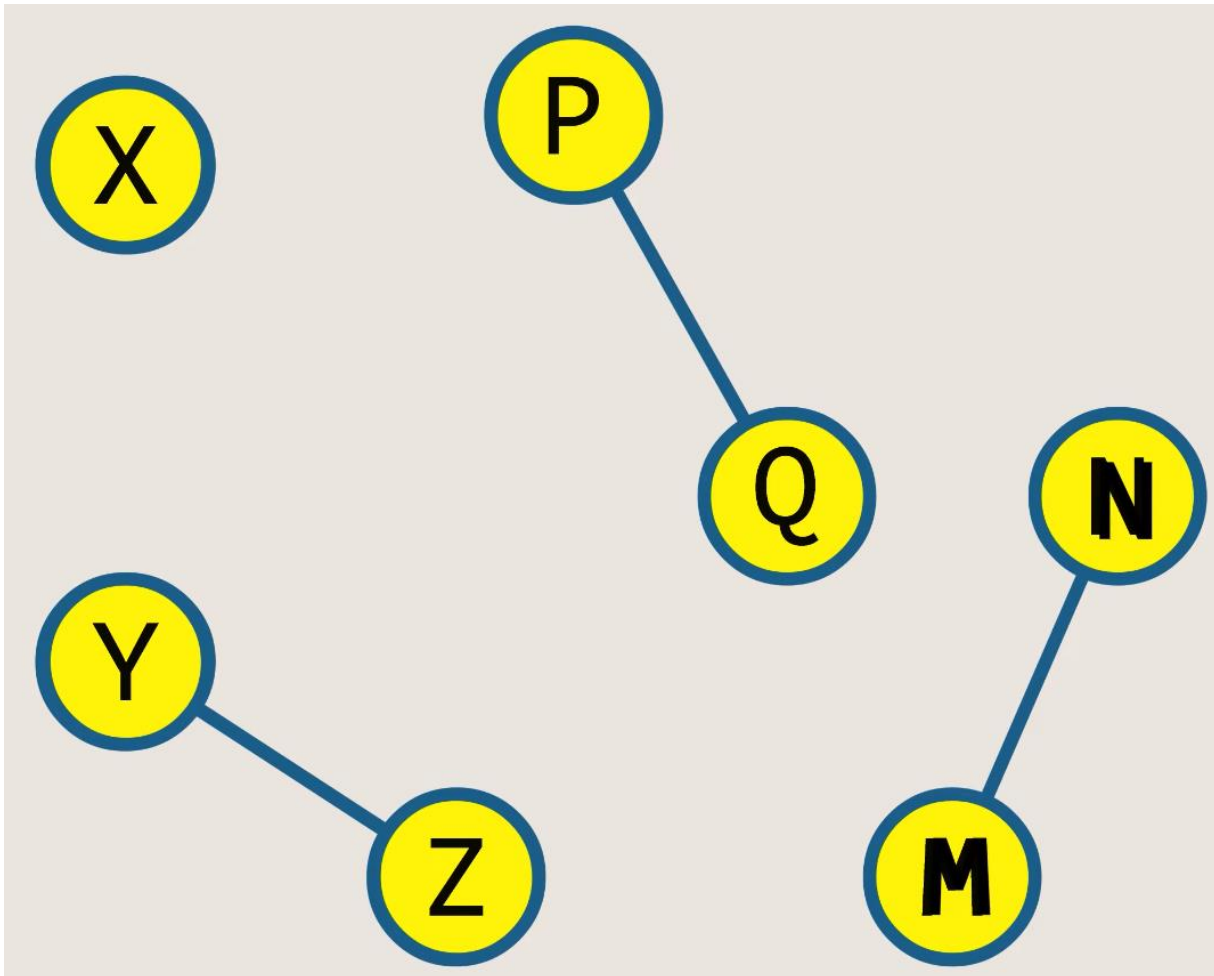
1 2 2 3 4 5 7

Kruskal's Algorithm



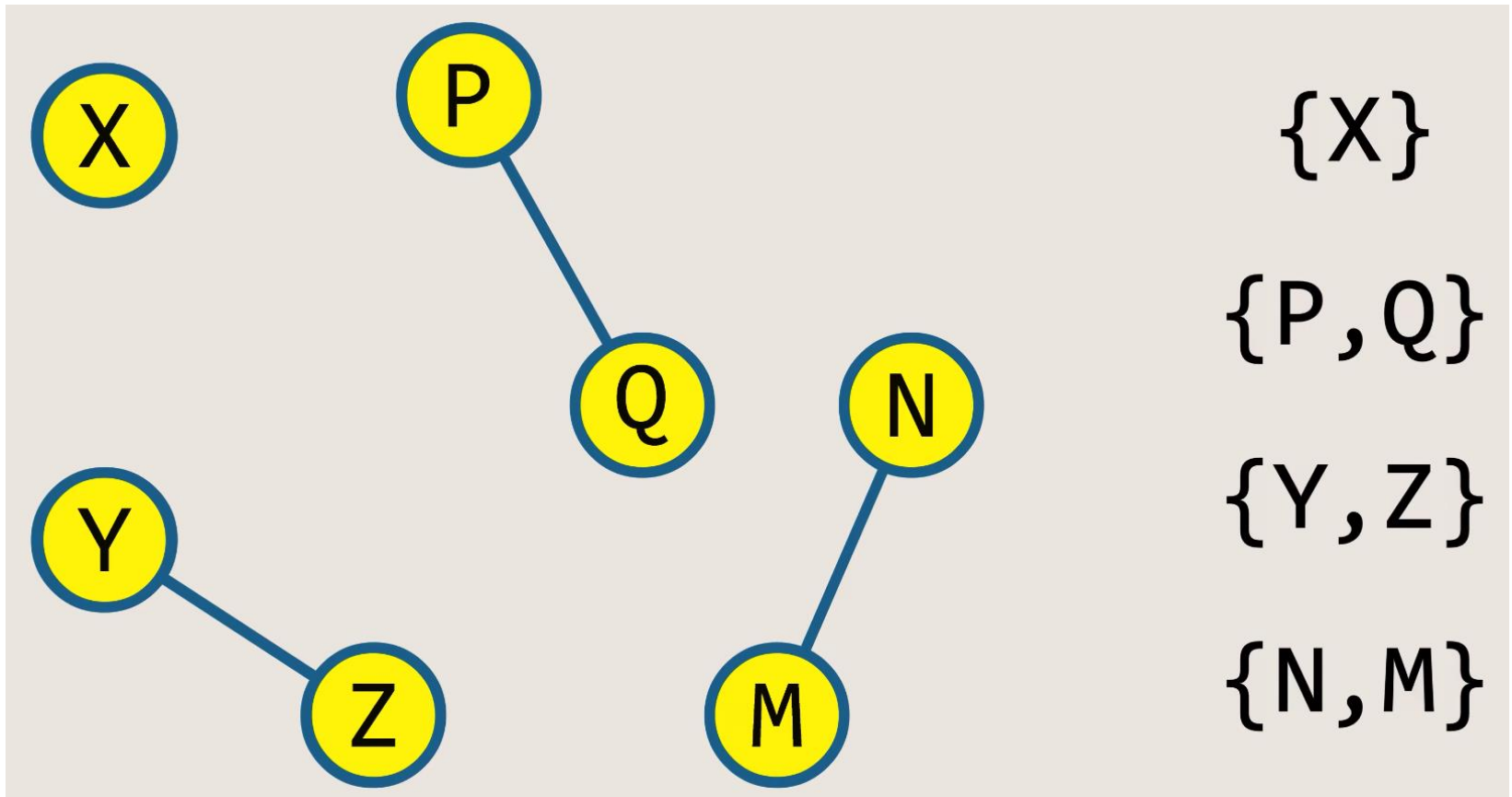
Step 6: All nodes are connected So algorithm stops.

Kruskal's Algorithm Cycle Detection



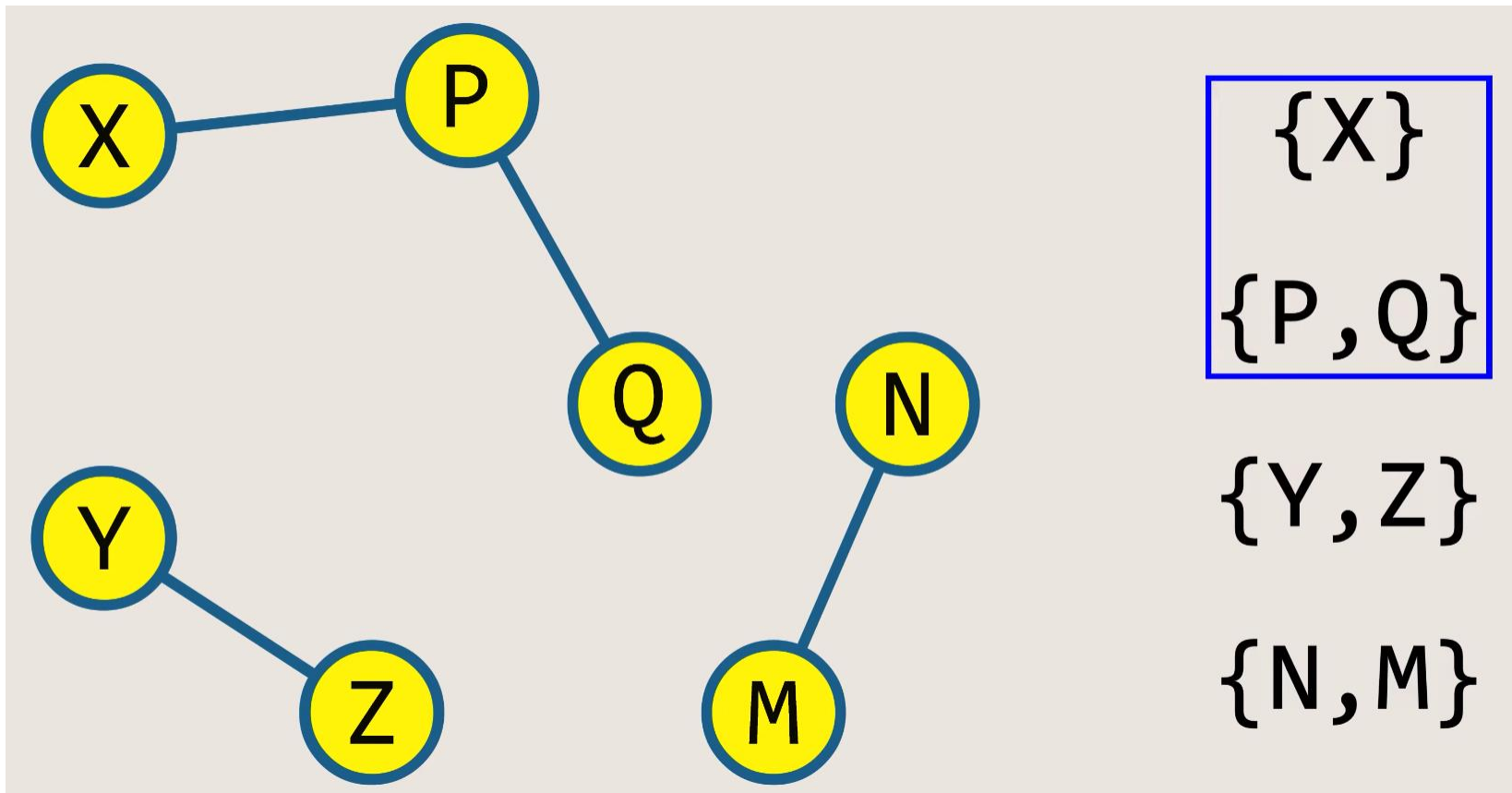
We have some
disconnected
graphs

Kruskal's Algorithm Cycle Detection



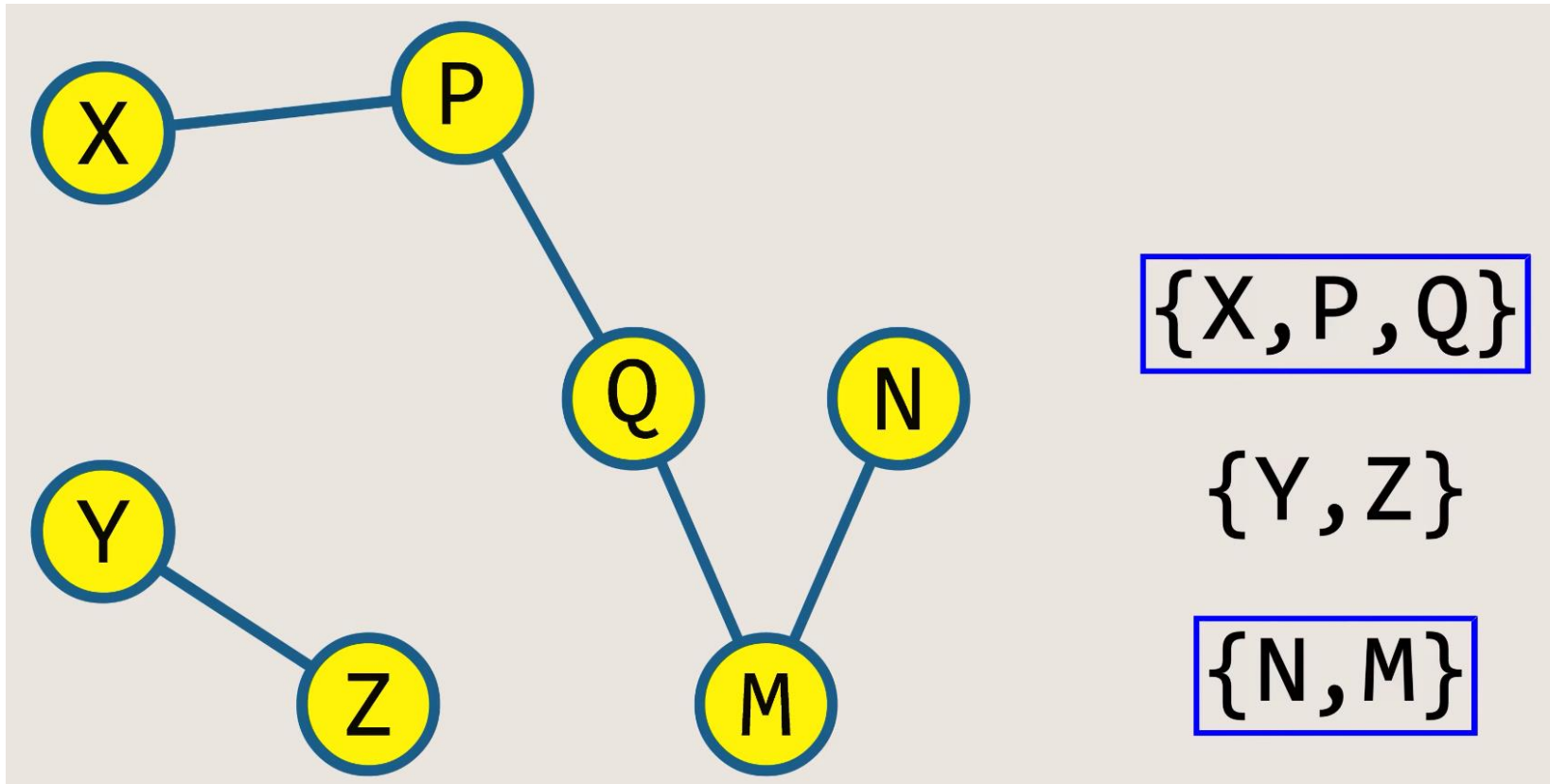
Write each graphs nodes as set

Kruskal's Algorithm Cycle Detection



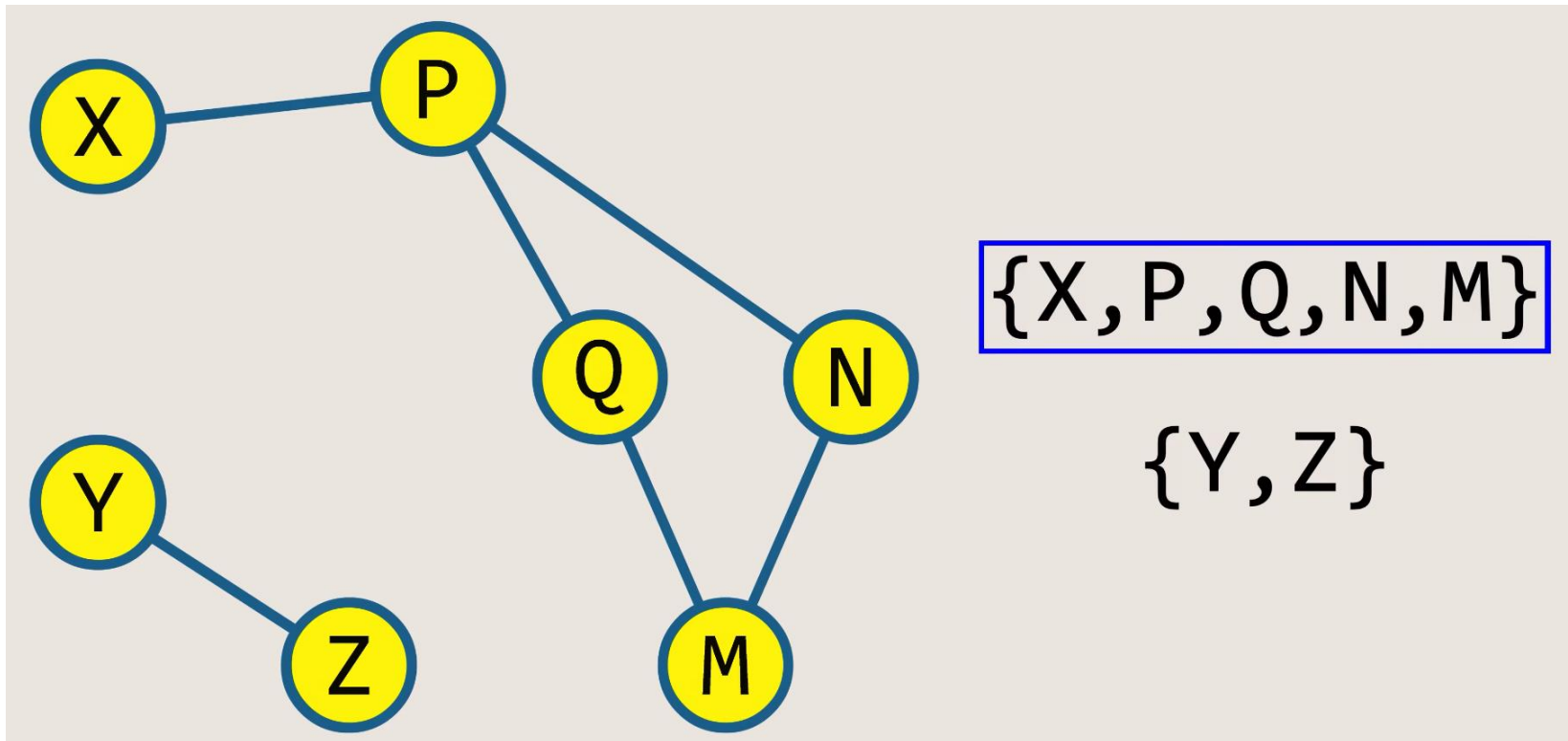
Connect Graph X to graph P, Q

Kruskal's Algorithm Cycle Detection



Connect Graph X, P, Q to graph M, N

Kruskal's Algorithm Cycle Detection



Now, let's try to connect P and N, but it forms a cycle, how do we know? Both P and N are present in the same set.

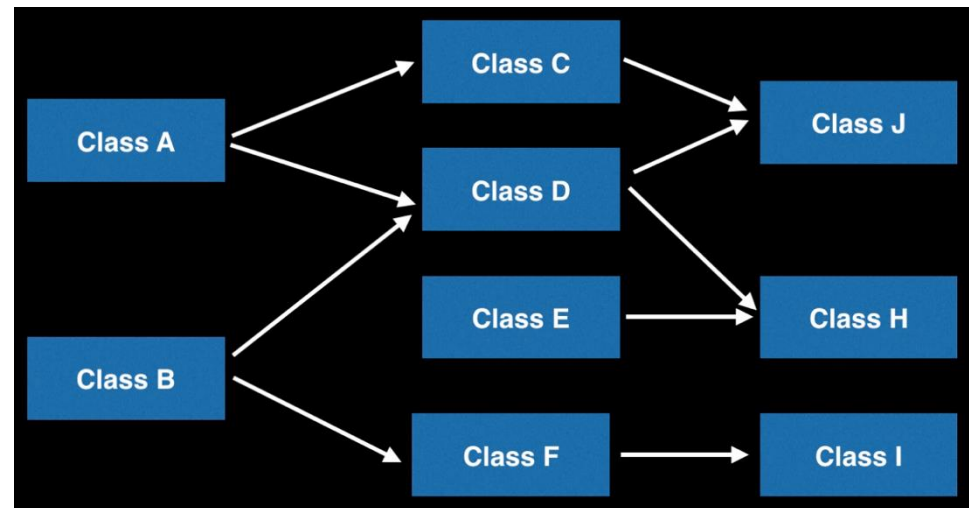
Topological Ordering

A **topological ordering** is: A linear ordering of vertices such that for every edge (u, v) , u comes before v .

Ordering Tasks with Dependencies

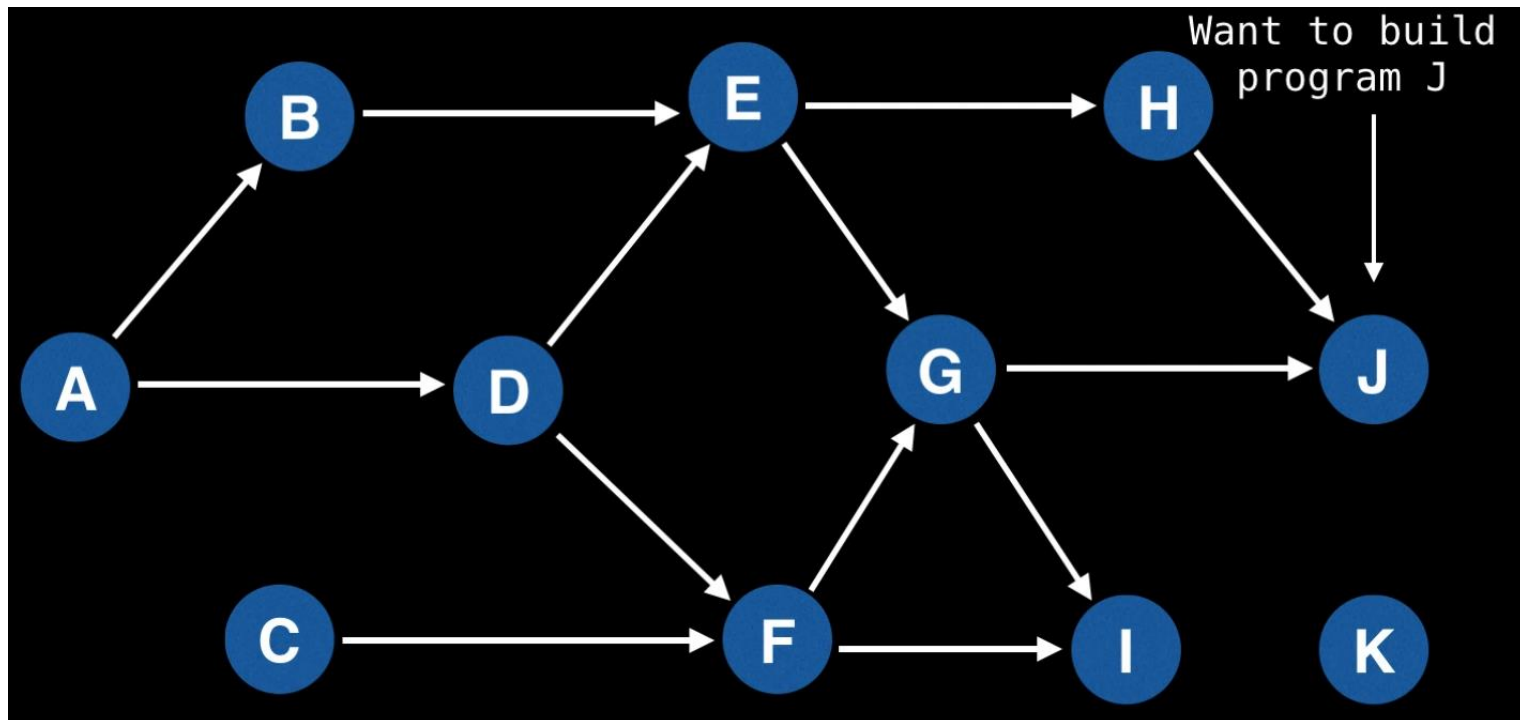
- Used in graphs
- Works only on **Directed Acyclic Graphs (DAGs)**
- Important for scheduling problems

Suppose you're a student at University X and you want to take Class H. Then you must first take classes A, B, D, and E as prerequisites. In this sense, there is an ordering on the nodes of the graph.



Topological Ordering

Another canonical example where an ordering on the nodes of the graph matters is program build dependencies. A program cannot be built unless its dependencies are built first.



Important Note

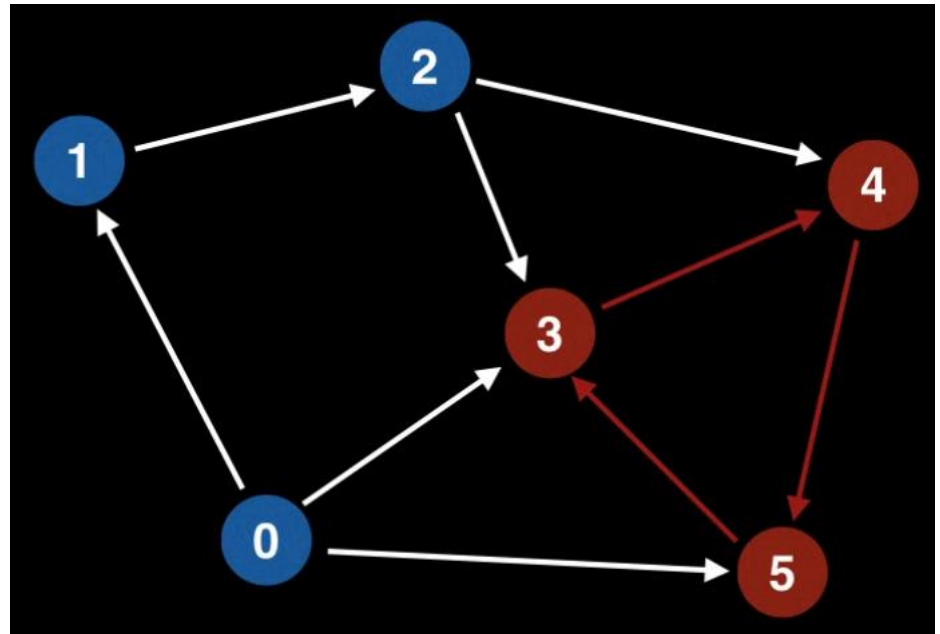
Directed Acyclic Graphs (DAG)

Not every graph can have a **topological ordering**.
A graph that contains a **cycle** cannot have a valid ordering.

Key Idea

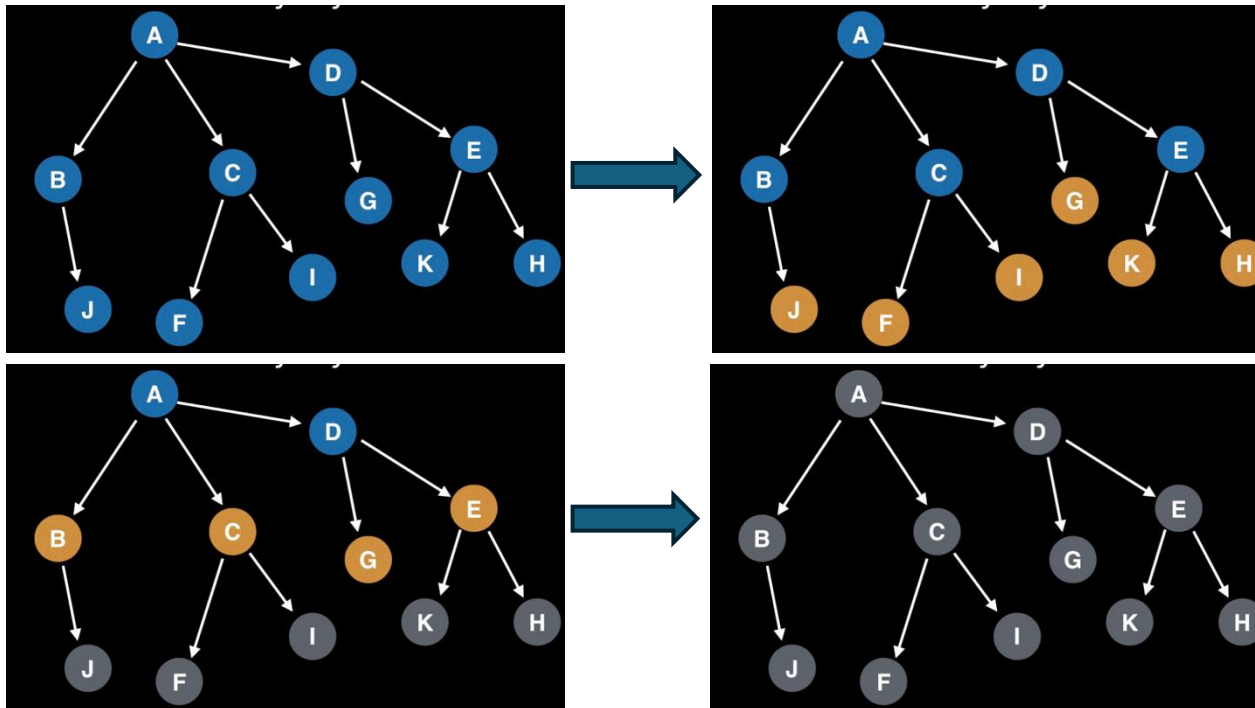
- Directed → edges have direction
- Acyclic → no cycles

Only DAGs can have a topological sort



Directed Acyclic Graphs (DAG)

By definition, all **rooted trees** have a topological ordering because they do not contain any cycles.



Topological Ordering (Left → Right)

A, C, D, E, B, G, H, K, J, F, I

Topological Sort

Topological Sort Using DFS as a result we use Stack data structure

Step 1

Pick an **unvisited node**

Step 2

Starting from the selected node, perform a

Depth First Search (DFS) exploring only **unvisited nodes**

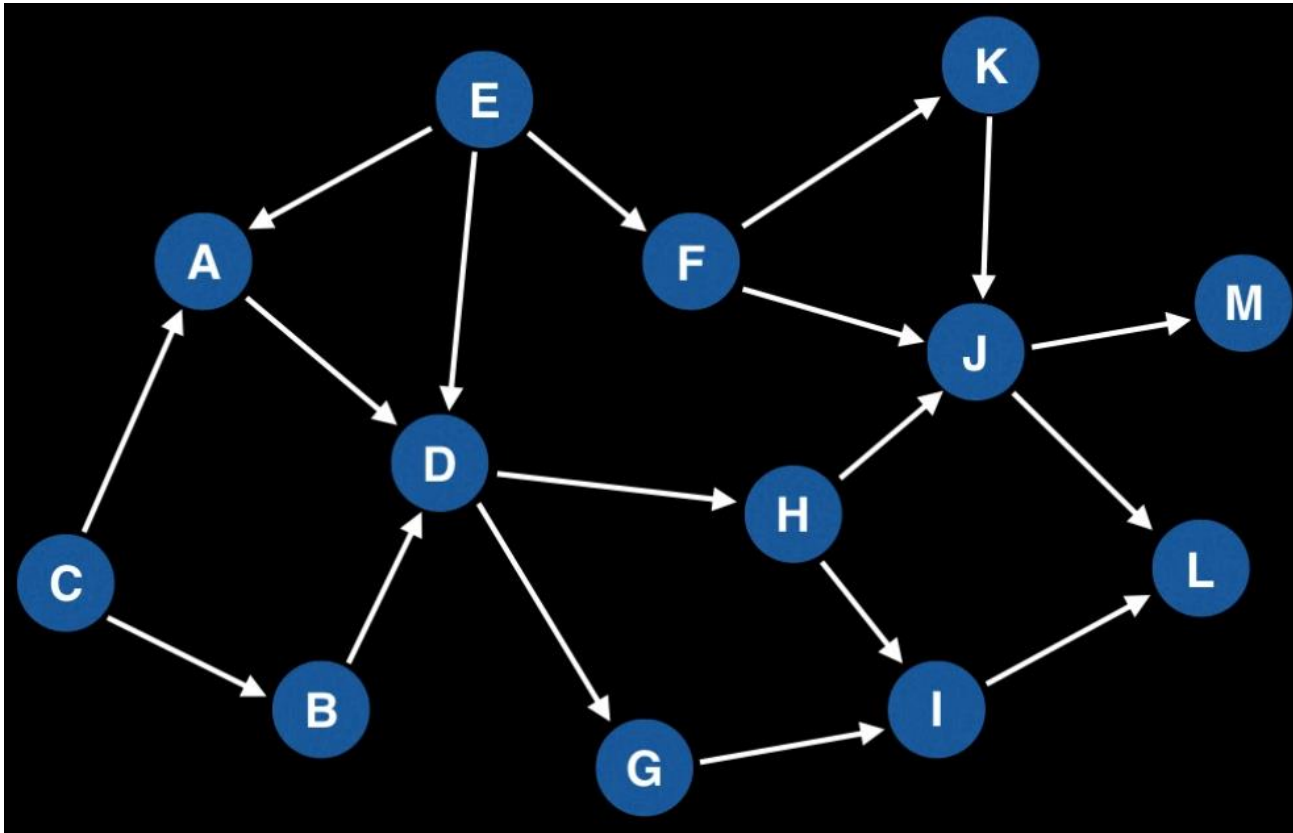
Step 3

On the **recursive return (backtracking)** of DFS:

Add the current node to the ordering

Insert it in **reverse order**

Topological Sort Example



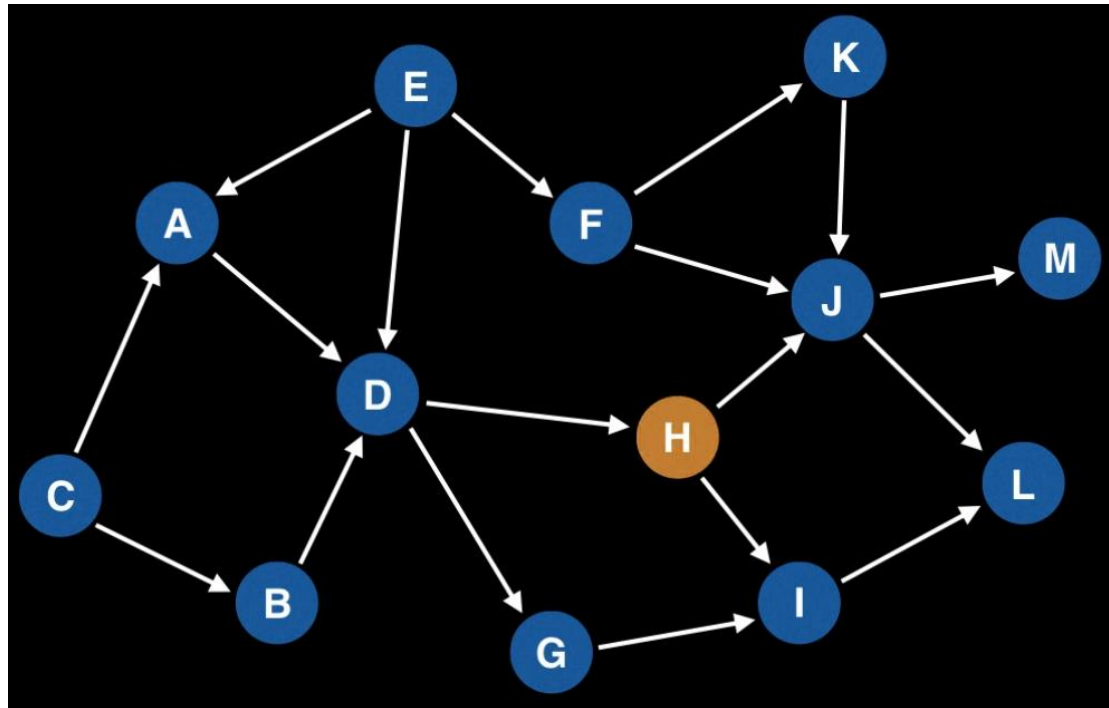
Topological ordering:

Topological Sort Example

DFS Recursion

Stack:

Node: H



Topological ordering:

Topological Sort Example

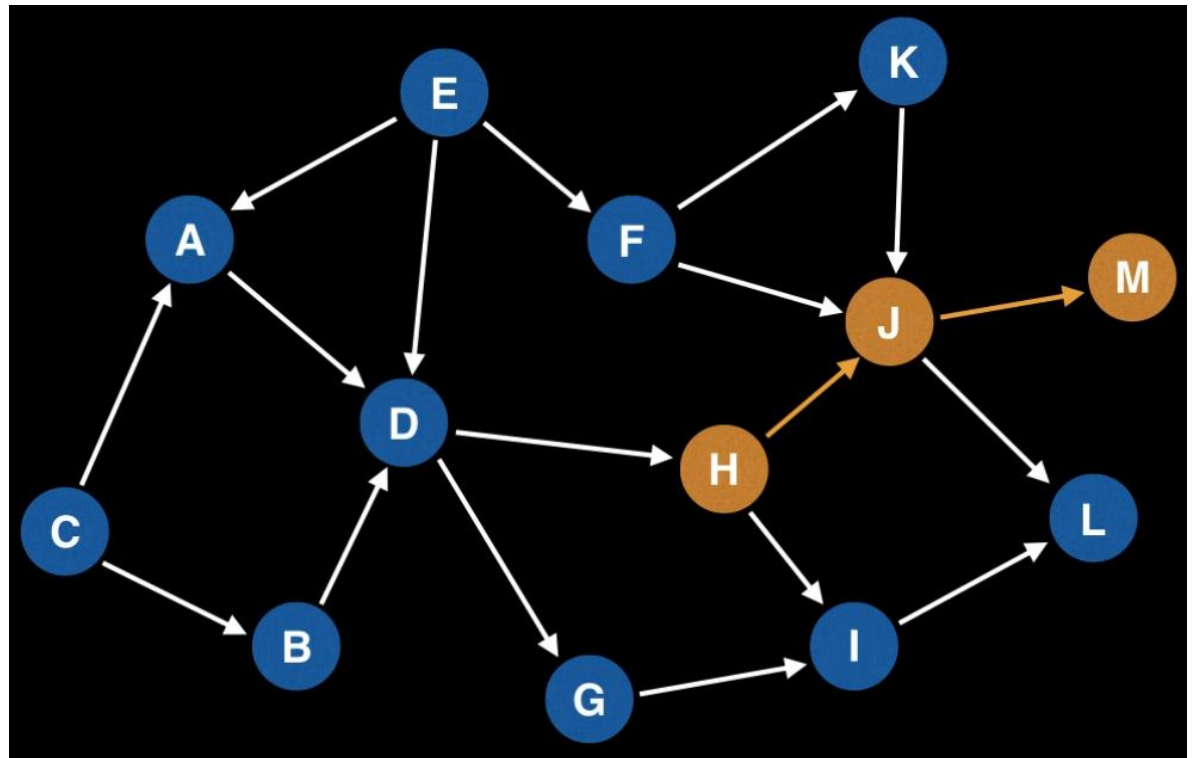
DFS Recursion

Stack:

Node: M

Node: J

Node: H



Topological ordering:

Topological Sort Example

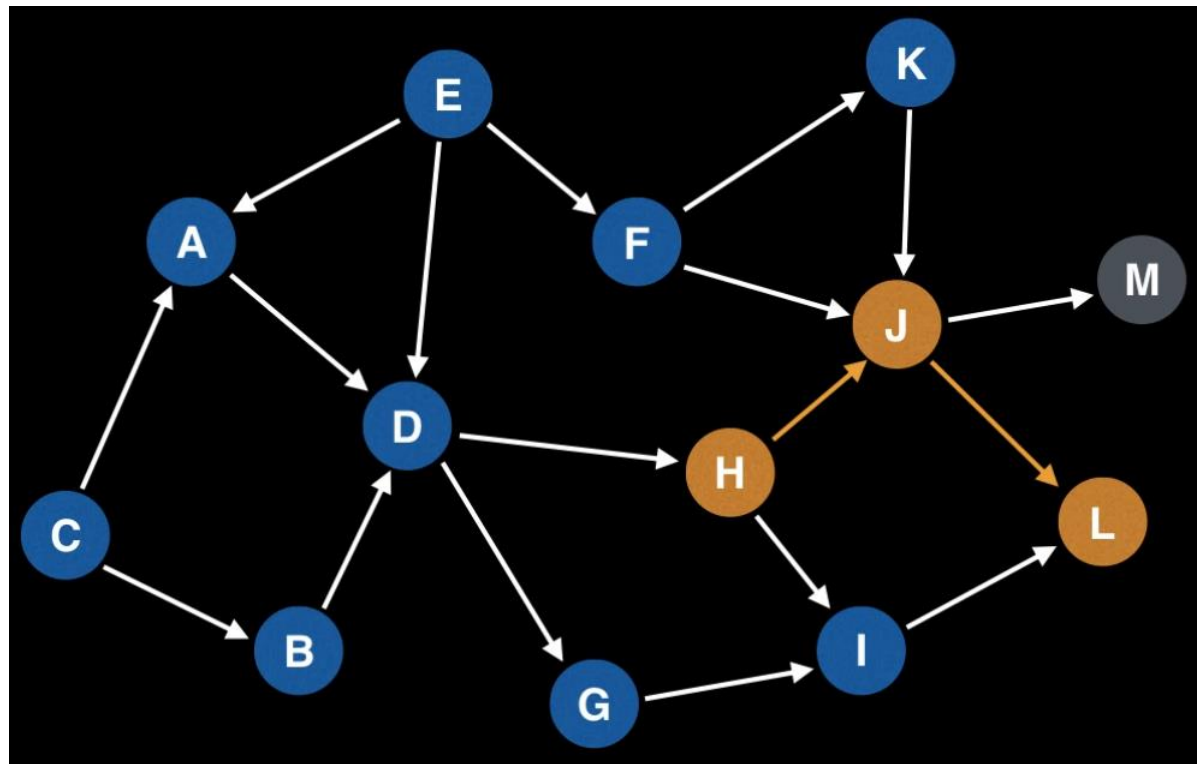
DFS Recursion

Stack:

Node: L

Node: J

Node: H



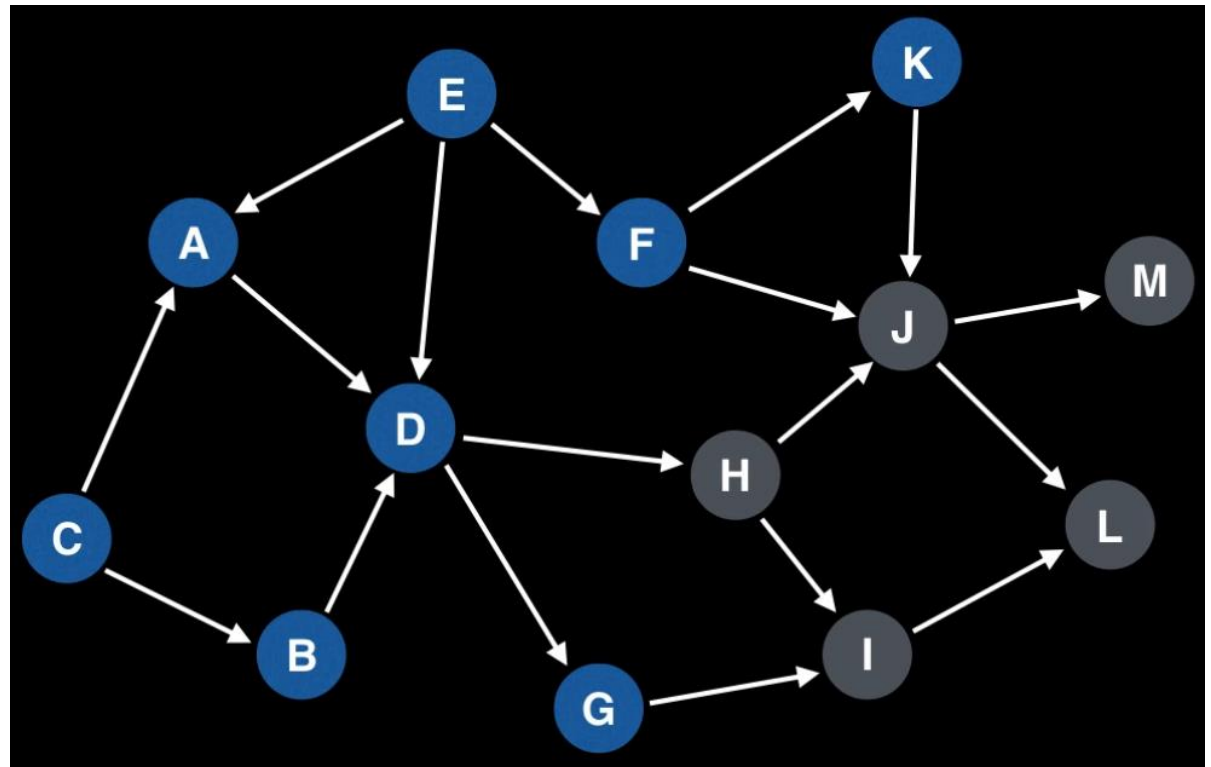
Topological ordering:

-----M

Topological Sort Example

DFS Recursion

Stack:



Topological ordering:

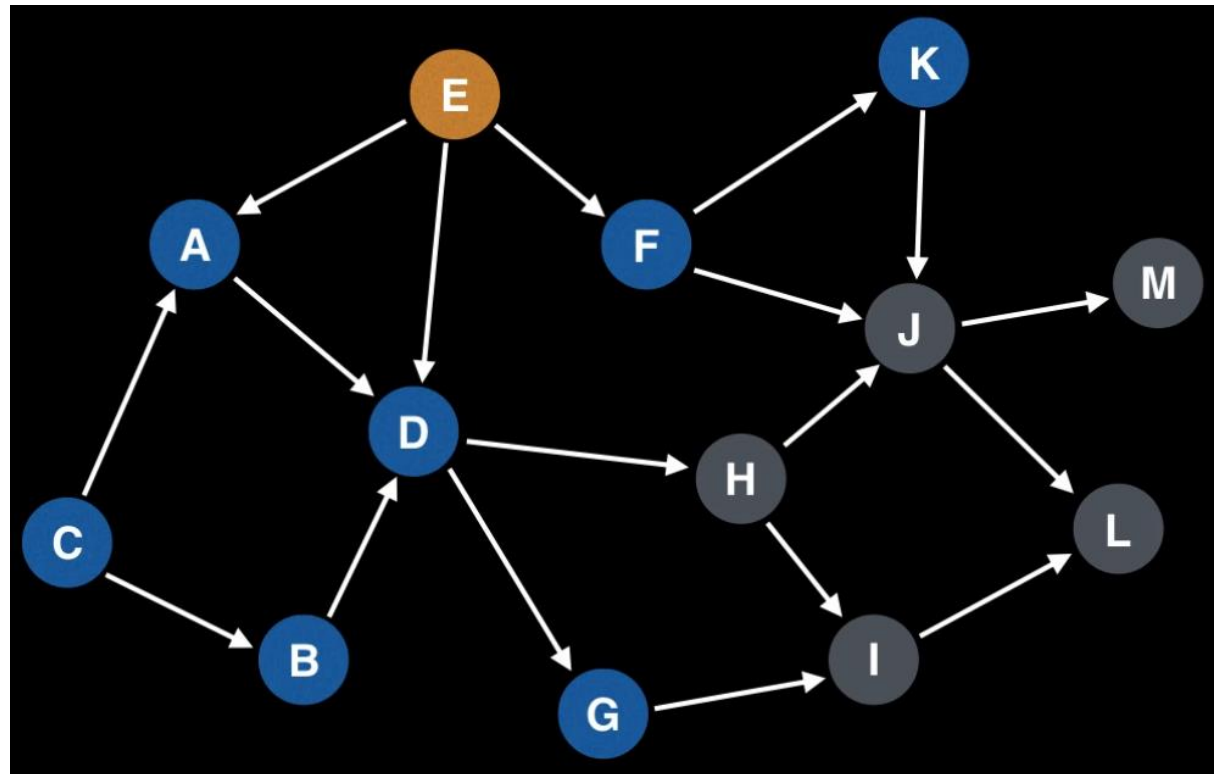
_____ H I J L M

Topological Sort Example

DFS Recursion

Stack:

Node: E

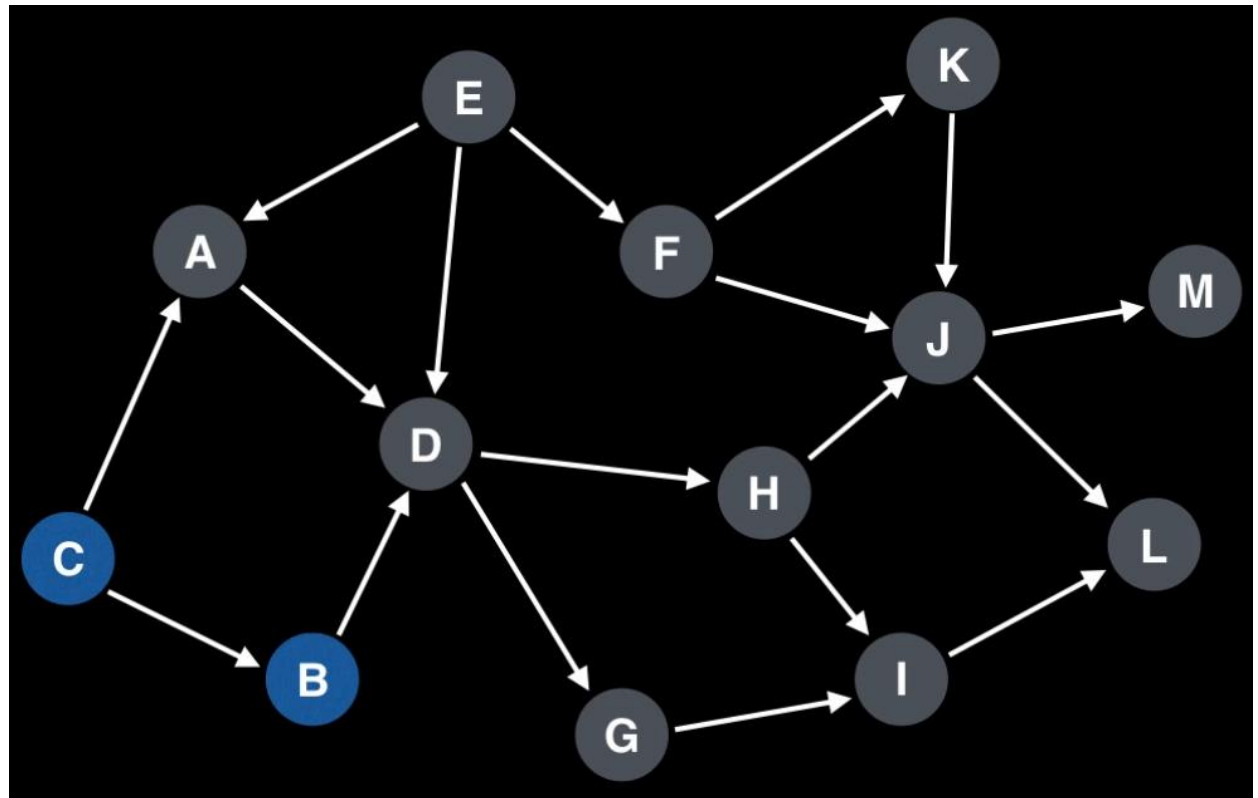


Topological ordering:

_____ H I J L M

Topological Sort Example

DFS Recursion
Stack:

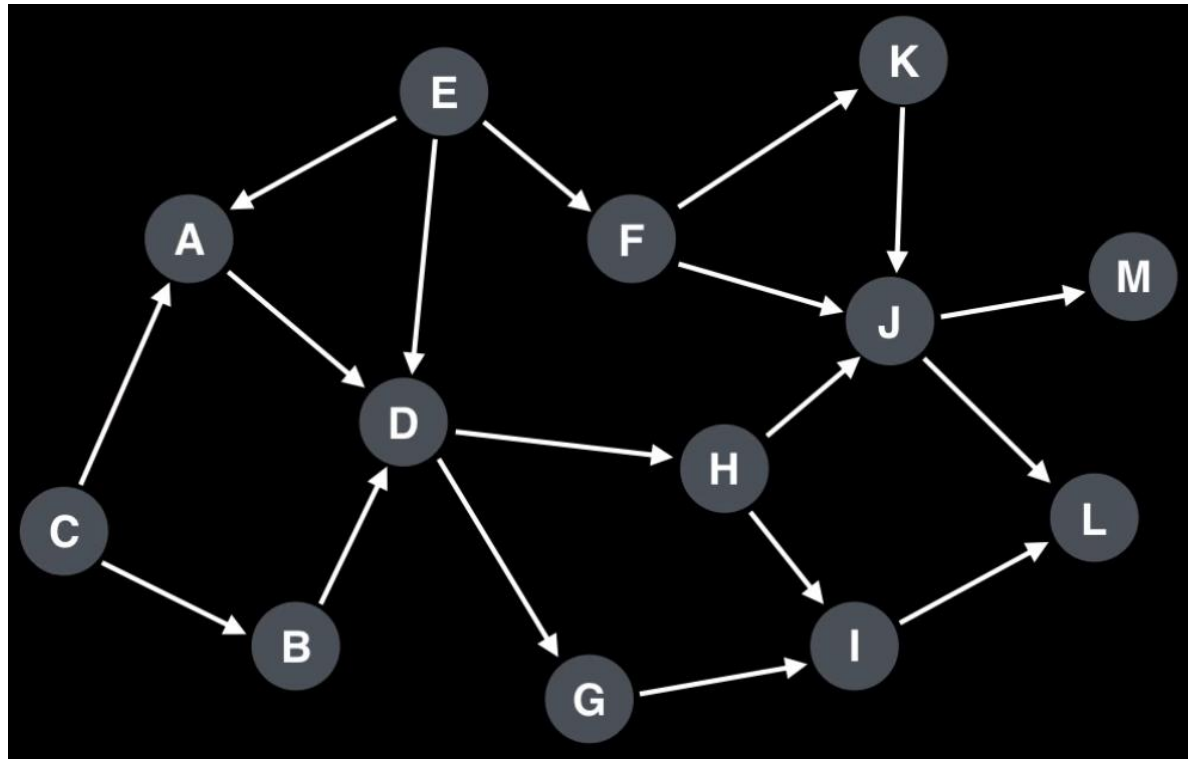


Topological ordering:
__E F K A D G H I J L M

Topological Sort Example

DFS Recursion

Stack:



Topological ordering:

C B E F K A D G H I J L M

Topological Sort Example (Kahn's Algorithm)

The intuition behind Kahn's algorithm is to **repeatedly remove nodes with no dependencies** from the graph and add them to the **topological ordering**.

As nodes without dependencies (and their outgoing edges) are removed from the graph, new nodes without dependencies will become available.

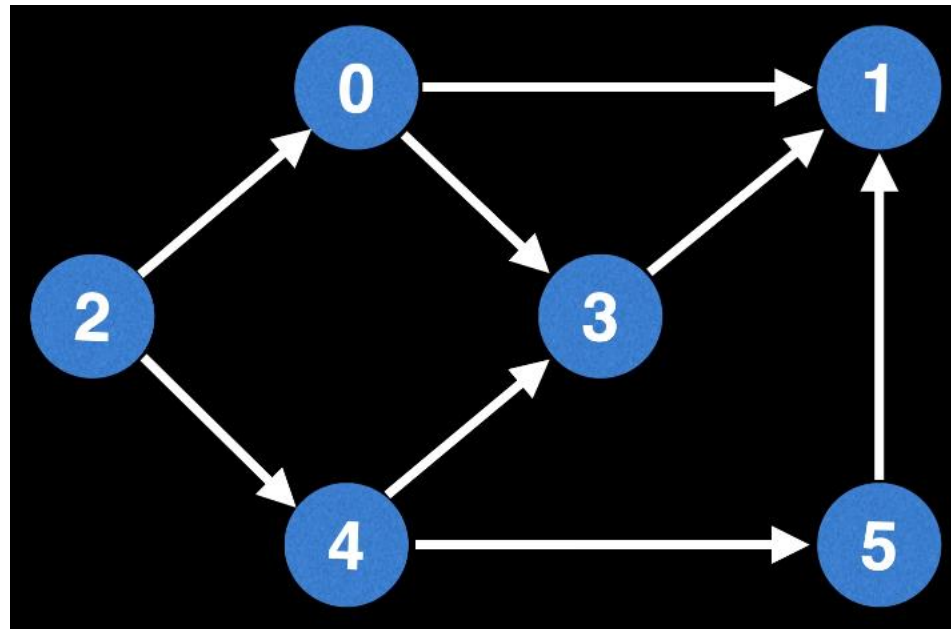
We continue removing nodes without dependencies until: all nodes are processed, **or a cycle is detected**.

We use Breadth-First Search (BFS) to perform Kahn's Algorithm. As a result, a queue is used to store the nodes with: "in-degree" = 0

Topological Sort Example

BFS Queue:

Node: 2



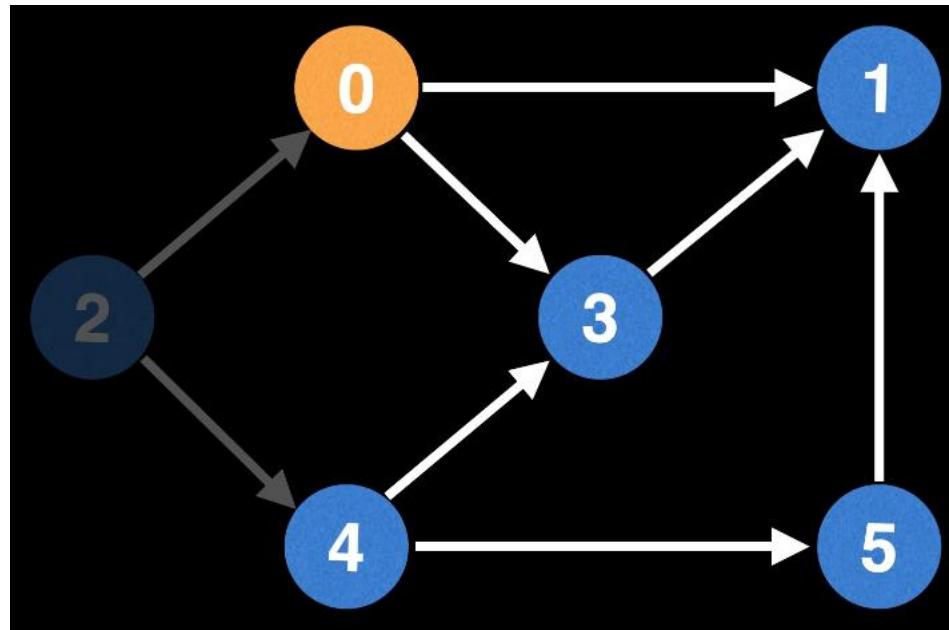
Topological ordering:

Topological Sort Example

BFS Queue:

Node: 4

Node: 0



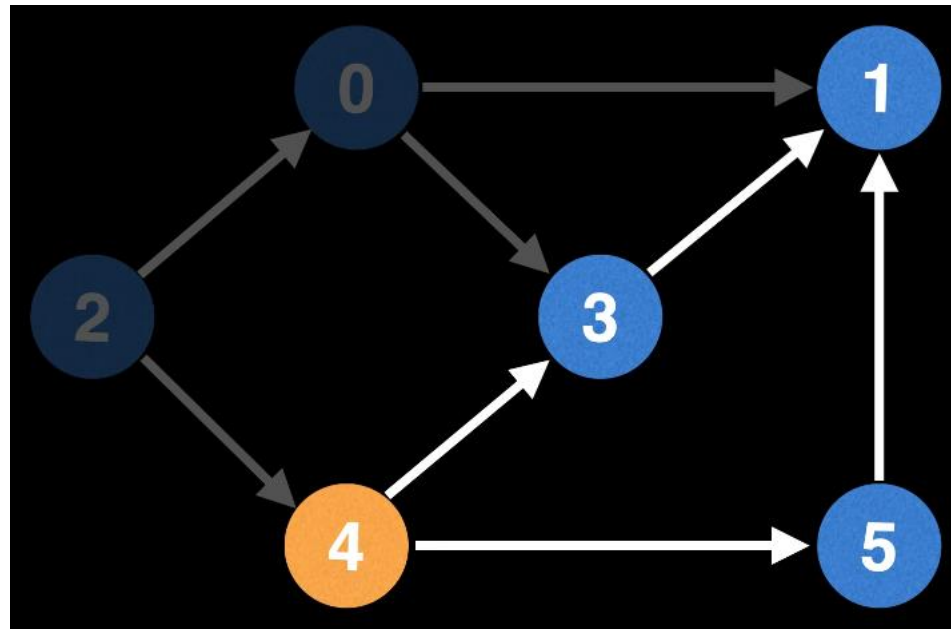
Topological ordering:

____2

Topological Sort Example

BFS Queue:

Node: 4



Topological ordering:

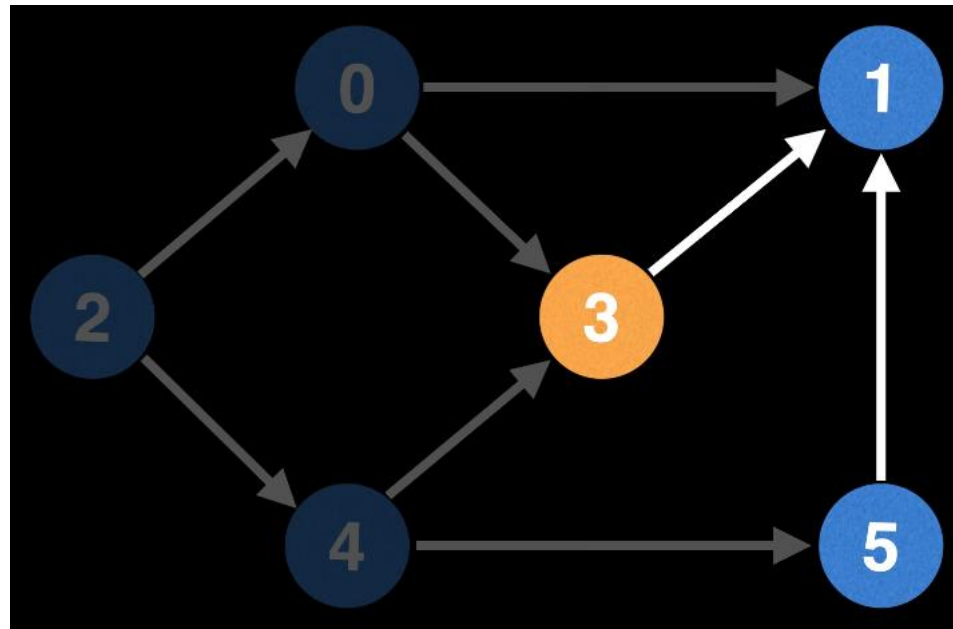
2 0 _ _ _ _

Topological Sort Example

BFS Queue:

Node: 5

Node: 3



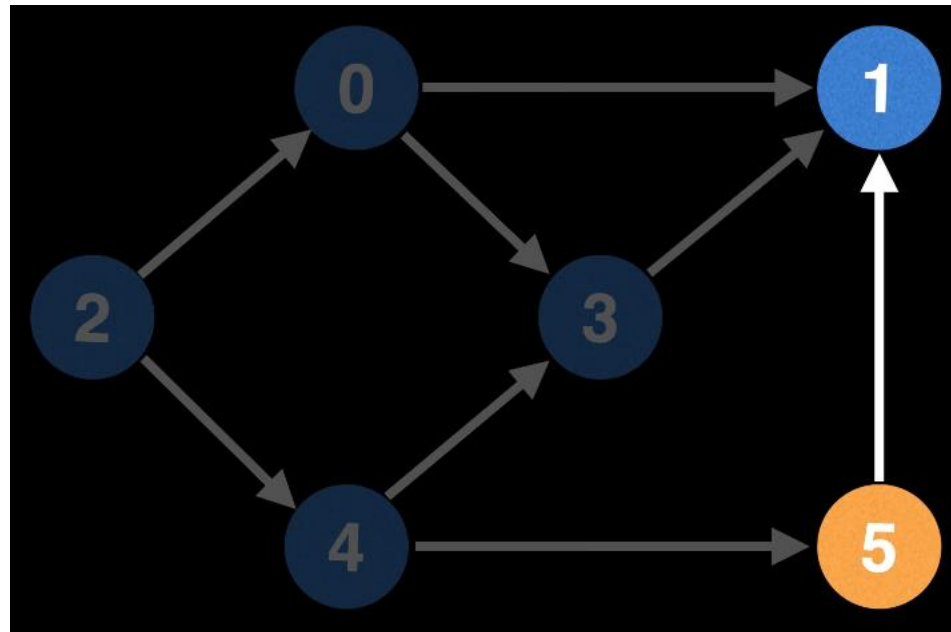
Topological ordering:

2 0 4 _ _ _

Topological Sort Example

BFS Queue:

Node: 5



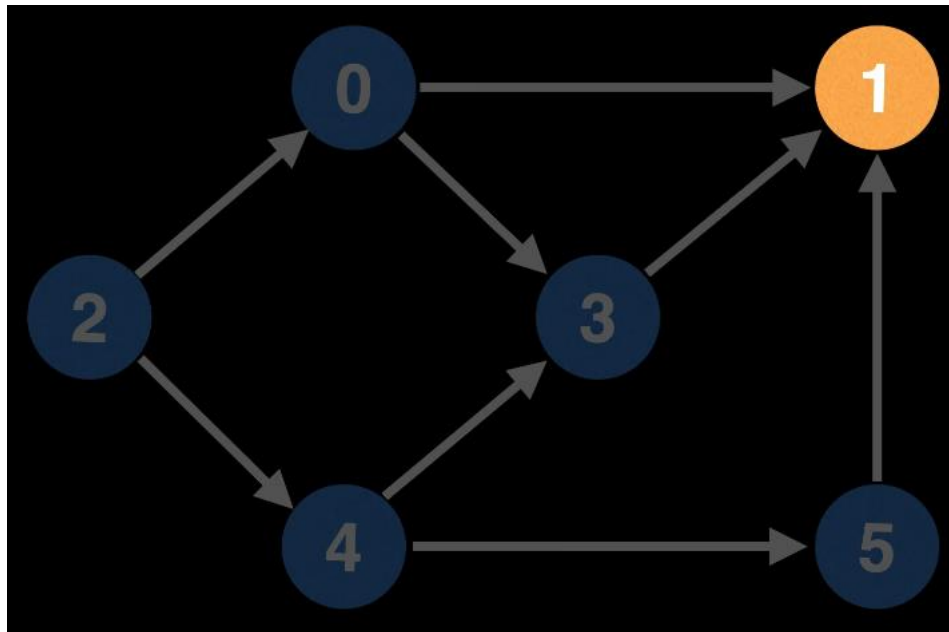
Topological ordering:

2 0 4 3 _ _

Topological Sort Example

BFS Queue:

Node: 1



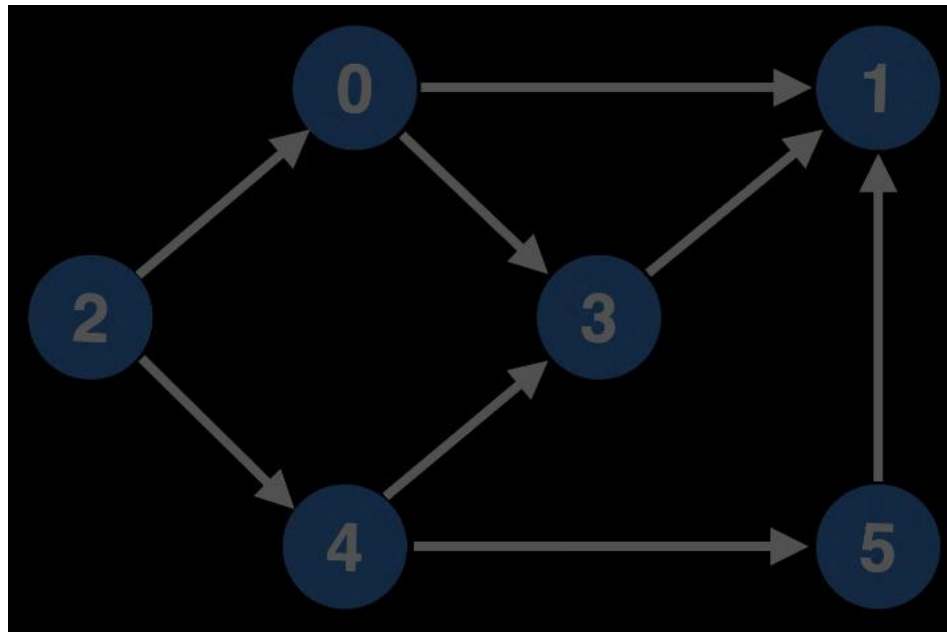
Topological ordering:

2 0 4 3 5 _

Topological Sort Example

BFS Queue:

Node: 1

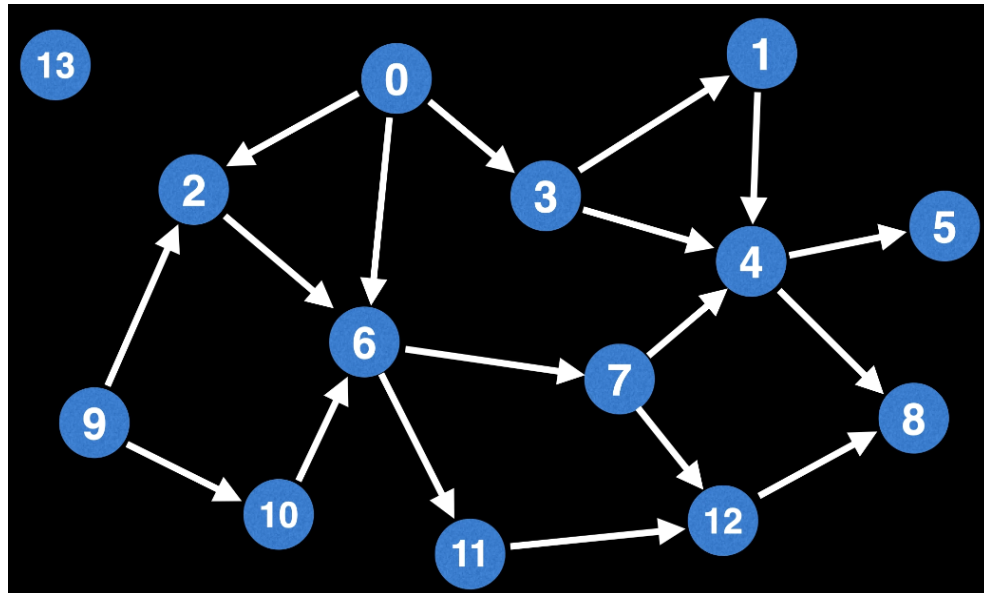


Topological ordering:

2 0 4 3 5 1

Topological Sort Example 2 (Assignment 4)

BFS Queue:



Nodes by incoming degree count:

0	1	2	3	4	5	6	7	8	9	10	11	12	13
0	1	2	1	3	1	3	1	2	0	1	1	2	0

Topological ordering:

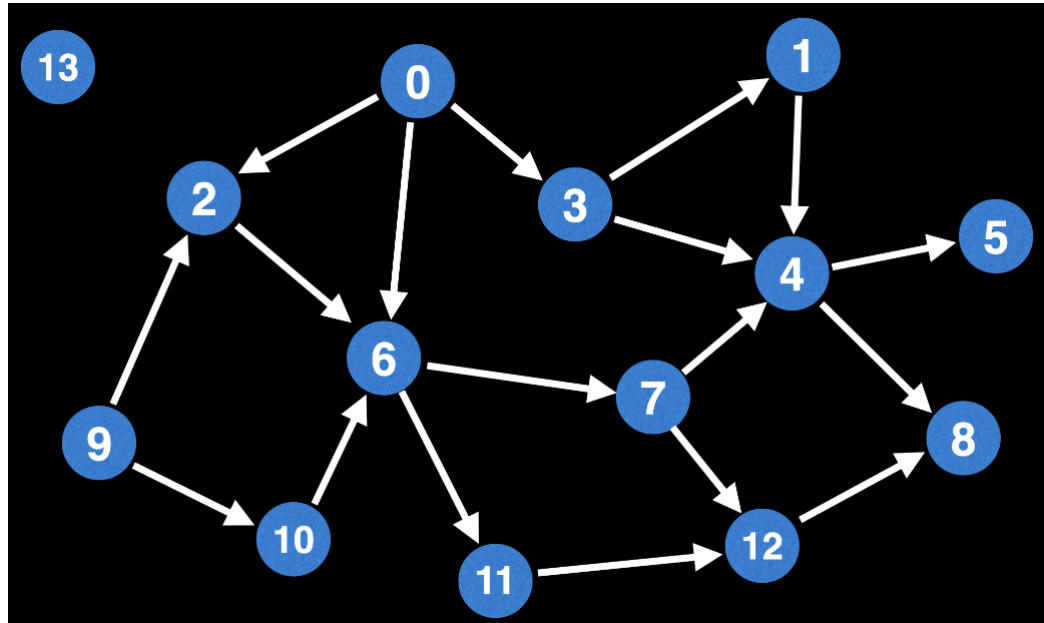
Topological Sort Example 2 (Assignment 4)

BFS Queue:

Node: 13

Node: 9

Node: 0



Nodes by incoming degree count:

0	1	2	3	4	5	6	7	8	9	10	11	12	13
0	1	2	1	3	1	3	1	2	0	1	1	2	0

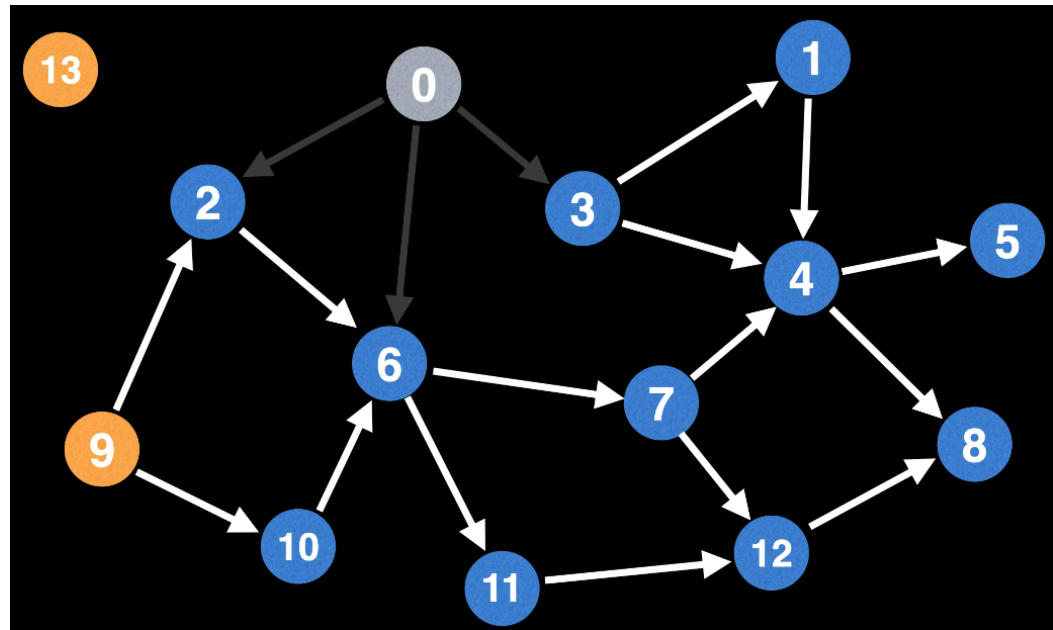
Topological ordering:

Topological Sort Example 2 (Assignment 4)

BFS Queue:

Node: 13

Node: 9



Nodes by incoming degree count:

0	1	2	3	4	5	6	7	8	9	10	11	12	13
0	1	1	0	3	1	2	1	2	0	1	1	2	0

Topological ordering:

0 _ _ _ _ _

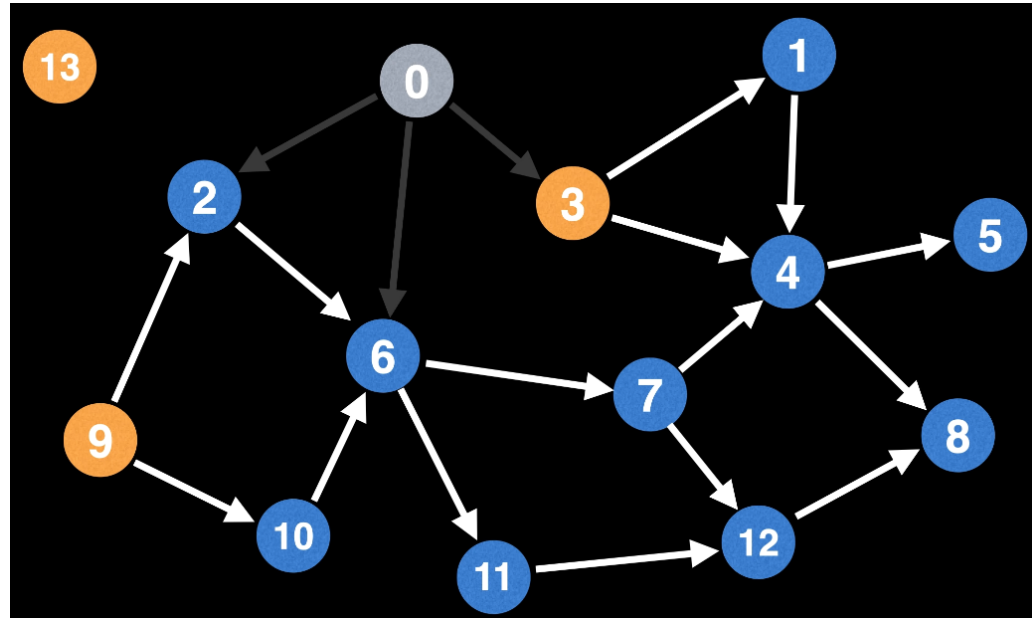
Topological Sort Example 2 (Assignment 4)

BFS Queue:

Node: 3

Node: 13

Node: 9



Nodes by incoming degree count:

0	1	2	3	4	5	6	7	8	9	10	11	12	13
0	1	1	0	3	1	2	1	2	0	1	1	2	0

Topological ordering:

0 _ _ _ _ _

Topological Sort Example 2 (Assignment 4)

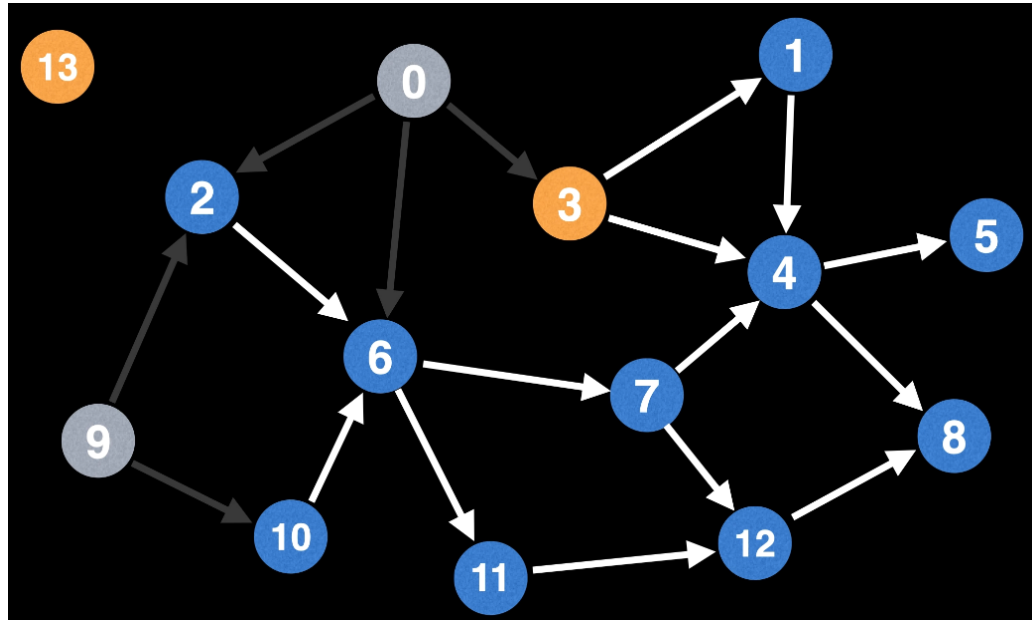
BFS Queue:

Node: 10

Node: 2

Node: 3

Node: 13



Nodes by incoming degree count:

0	1	2	3	4	5	6	7	8	9	10	11	12	13
0	1	0	0	3	1	2	1	2	0	0	1	2	0

Topological ordering:

0 9 _ _ _ _ _